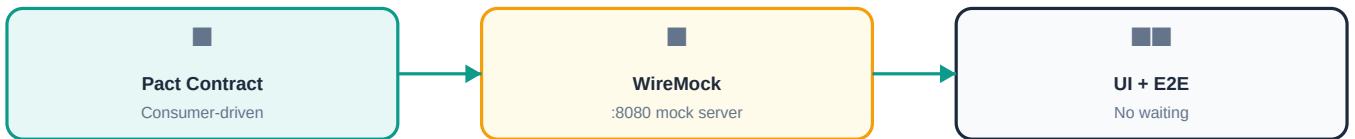


API Contract Testing and Mocking Starter Kit for SDETs

Test UI Against Mocks While Backend Is Still Being Built

- ◆ 5 Pact.js contract test templates • Pact V3 + MatchersV3
- ◆ 3 WireMock setup scripts • JAR • Docker • Programmatic
 - ◆ Postman collection (v2.1) with pm.test assertions
- ◆ 10-page breaking-change checklist • 14 categories
- ◆ UI-against-mock hero workflow • 10 steps, 2 terminals



@shamshutalks • Free & Open-Source Stack • No Vendor Lock-In
Pact 17.1.3 • WireMock 3.13.2 • Jest 29 • Express 4 • Postman v2.1 • Node 18+

What You'll Learn

By the end of this kit you will be able to **write consumer-driven contracts with Pact**, **spin a WireMock mock server in one command**, **validate mocks with Postman**, and **test your UI before the backend exists** — with a checklist that catches breaking changes before they reach production.

- **Contract testing methodology** — stub vs mock vs contract, consumer-driven vs provider-driven, when to use which
- **Pact.js V3 matchers** — like / regex / eachLike / integer / fromProviderState and what drift each catches
- **WireMock mock server** — JAR, Docker, Compose, mappings, Handlebars templating, record/playback
- **Postman validation** — collection v2.1, pm.test + pm.expect, collection variables, chaining
- **UI-against-mock workflow** — 10-step sequence, env var switch, CORS, proxy fallback, Playwright E2E
- **Breaking-change prevention** — 14 categories, severity matrix, decision tree, printable quick-ref

What's Included — Every File on Disk

Every path below matches a real file in your download. No hallucinated paths.

#	Item	Type	Path on Disk	What You Get
1	Pact Template 01 — GET /users/:id	Code template	companion_files/pact-templates/template-01-consumer-contract-basic.test.js	Path param + type matchers
2	Pact Template 02 — POST /users	Code template	companion_files/pact-templates/template-02-consumer-contract-create.test.js	Body + regex + eachLike
3	Pact Template 03 — GET /users?search	Code template	companion_files/pact-templates/template-03-consumer-contract-search.test.js	Query params + array
4	Pact Template 04 — Error Handling	Code template	companion_files/pact-templates/template-04-consumer-contract-error-handling.test.js	404/422 + nullValue
5	Pact Template 05 — Provider Verify	Code template	companion_files/pact-templates/template-05-provider-verification.test.js	Verifier + stateHandlers
6	WireMock JAR	Script	companion_files/wiremock-setup/01-stand-alone-jar.sh/.bat	java -jar + CORS + templating
7	WireMock Docker	Script	companion_files/wiremock-setup/02-docker.sh/.bat + docker-compose.yml	docker run / compose
8	WireMock Mappings	Config JSON	companion_files/wiremock-setup/03-mappings/mappings/*.json	3 mappings + __files
9	Postman Collection	Collection v2.1	companion_files/postman/Contract_Mock_Validation.postman_collection.json	3 folders, 6 requests
10	Breaking-Change Checklist	Checklist	companion_files/checklists/api-breaking-change-checklist.md	14 categories + tree
11	Demo App + Practice Lab	Demo / Lab	companion_files/contract-mock-demo-app/+ practice-lab/	Express 5 endpoints + 5 exercises

Start Here

New to contract testing? Start at **Ch 1 Methodology (p6)**. Want templates now? Jump to **Ch 3 Pact Templates (p12)**. Need mock server now? Jump to **Ch 4 WireMock (p20)**. All chapters link back to the 6-stage path on p5.

Project Structure — Exact Folder Tree on Disk

This is the exact tree you will see after unzipping. Every path in this PDF matches this tree.

```
Project Tree — matches disk exactly tree

API_Contract_Testing_and_Mocking_Starter_Kit_for_SDETs/
  README.md # 10-point onboarding guide
  requirements.txt # Node + WireMock + Postman + ReportLab
  .env.example # PORT, WIREMOCK_URL, PACT_DIR
  companion_files/
    pact-templates/ # 5 Pact.js templates (PactV3 + MatchersV3)
      template-01-consumer-contract-basic.test.js
      template-02-consumer-contract-create.test.js
      template-03-consumer-contract-search.test.js
      template-04-consumer-contract-error-handling.test.js
      template-05-provider-verification.test.js
    wiremock-setup/ # 3 WireMock setups
      01-standalone-jar.sh / .bat
      02-docker.sh / .bat
      docker-compose.yml
      wiremock-03-start-mock-server.js
      03-mappings/mappings/
        get-user.json
        create-user.json
        search-users.json
      __files/user-1.json
    postman/
      Contract_Mock_Validation.postman_collection.json
    checklists/
      api-breaking-change-checklist.md
    contract-mock-demo-app/ # Runnable Express provider
      server.js • package.json • data/users.json
      tests/ (mirrors pact-templates)
    contract-mock-practice-lab/ # 5 exercises + helpers
      helpers/ (pact-helpers, wiremock-helpers, data-factory)
      exercises/01-05 (each: README + exercise.test.js + solution/)
  API_Contract_Testing_and_Mocking_Starter_Kit.pdf # this guide (35 pages)
```

Pre-requisites — Free / Open-Source Stack Only

Every tool is free/open-source. You need only **ONE of Java or Docker** for WireMock.

Tool	Version	Why Needed	Download Link	Verify Command
Node.js	18+ (20 LTS rec.)	Runs Pact, Express, Jest	nodejs.org	node -v → v20.x
Java (Temurin)	17 LTS (11+ min)	WireMock JAR only	adoptium.net	java -version → 17.x
Docker Desktop	Latest	WireMock without Java	docker.com	docker --version → 24.x
Postman	Latest (free tier)	Validates mock responses	postman.com	Open Postman → About
VS Code	Latest	Editor for templates	code.visualstudio.com	code --version
Git	Latest	Version pact files	git-scm.com	git --version

Java vs Docker — You Need Only One

Have Docker? Use Docker — no Java needed. No Docker? Use JAR — needs Java 11+ (Temurin 17 recommended). Team sharing? Use **docker-compose.yml**. All three produce the same mock server at **http://localhost:8080**.

Windows vs Mac vs Linux — where commands differ, both variants are shown in each chapter (e.g., volume mounts: -v \$PWD:/home/wiremock on Mac/Linux vs -v \${PWD}:/home/wiremock on PowerShell).

Table of Contents

Click any entry to jump — hyperlinked PDF bookmarks.

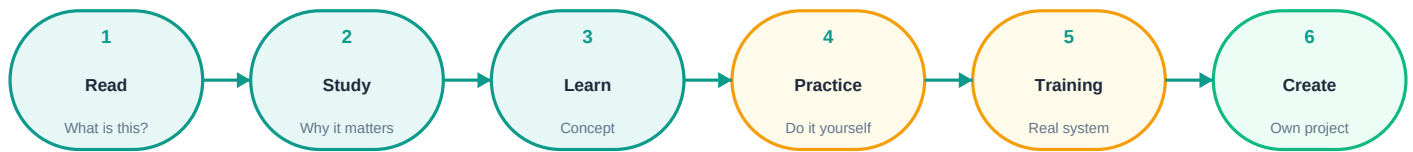
Section	Title	Page
FM	Cover • What You'll Learn • What's Included	1–2
FM	Project Structure • Pre-requisites	3
FM	Table of Contents (you are here)	4
FM	How to Use This Kit — 6-Stage Learning Path	5
Ch 1	Methodology — Why Contract Testing Matters	6–9
1.1	The Problem: Backend Breaks Frontend	6
1.2	Stub vs Mock vs Contract — Know the Difference	7
1.3	Consumer-Driven vs Provider-Driven	8
1.4	Pact + WireMock Together — Why Both?	9
Ch 2	Contract Testing Lifecycle	10–11
2.1	The 6-Step Consumer-Driven Loop	10
2.2	Pact File Anatomy + can-i-deploy Gate	11
Ch 3	Pact.js Templates — Five Ready-to-Use Tests	12–19
3.0	PactV3 Setup + Matchers Reference (T3)	12
3.1	Template 01: GET /users/:id	13
3.2	Template 02: POST /users	14
3.3	Template 03: GET /users?search	15
3.4	Template 04: Error Handling 404/422	16
3.5	Template 05: Provider Verification	17
3.6	Running All 5 + Pact File Output	18
3.7	Common Pitfalls + Matcher Cheat Sheet	19
Ch 4	WireMock — Three Setup Scripts	20–24
4.1	Setup Options Comparison (T5)	20
4.2	Script 01: Standalone JAR	21
4.3	Script 02: Docker + Compose	22
4.4	Script 03: Mappings + __files	23
4.5	Response Templating + Record/Playback	24
Ch 5	Postman Collection — Validate Mock Responses	25–26
5.1	Collection Structure + Variables	25
5.2	pm.test Assertions + Validation Workflow	26
Ch 6	UI Against Mock — Exact Setup While Backend Builds (HERO)	27–29

Section	Title	Page
6.1	End-to-End Sequence — The Full Workflow	27
6.2	Env Var Switch + CORS + Proxy Fallback	28
6.3	Playwright E2E Against WireMock + Verification	29
Ch 7	Breaking-Change Checklist — Catch Schema Drift Early	30–33
7.1	How to Use the Checklist — Severity Legend	30
7.2	Request Breaking Changes	31
7.3	Response Breaking Changes	32
7.4	Decision Tree + Printable Quick-Ref	33
App	Troubleshooting — 8 Common Issues (T8)	34
App	Glossary — 20 Terms • What's Next • Branding	35

How to navigate: **Ctrl/Cmd + click** any TOC entry to jump. Bookmarks panel (left in Acrobat/Preview) also lists all chapters. Every chapter starts with a **SectionDivider** showing chapter number and title.

How to Use This Kit — 6-Stage Learning Path

Every file in this kit supports all 6 stages. Follow this path from zero to creating your own contract suite.



Stage	Question	What You Do	Where in Kit
1. Read	What is this?	Read cover + What's Included + Start Here	p1–p5 + README.md
2. Study	Why does it matter?	Study diagrams + tables + architecture	p6–p11, D1–D9, T1–T9
3. Learn	What concept do I now understand?	Learn matchers + lifecycle + glossary	p12, p35 Glossary 20 terms
4. Practice	How do I do it myself?	Run 5 Pact templates + 3 WireMock scripts	p12–p26 + companion_files/
5. Training	Can I train on a real system?	Run demo-app + WireMock + Postman + lab	p27–p29 + demo-app + lab
6. Create	Can I create my own after this?	Fork demo-app, add endpoint, write Pact test	p35 What's Next + helpers

Before You Start — Checklist

■ Node 18+ (node -v) • ■ Terminal open (PowerShell / Terminal) • ■ Internet • ■ Files at
API_Contract_Testing_and_Mocking_Starter_Kit_for_SDETs/ • ■ VS Code

If any fails, see **p3 Pre-requisites** for download links + verify commands. You need only ONE of Java or Docker for WireMock.

Learning path in one line: **Read p1–p5 (2 min)** → **Study p6–p11 (diagrams)** → **Learn p12 matchers** → **Practice p13–p26 (templates + mocks)** → **Training p27–p29 (UI workflow)** → **Create p35 (fork & extend)**. Every instructional section has a 7-part flow: What / What-Needed / Where / How / What-You-Should-See / What-If / What's-Next.

01

Methodology

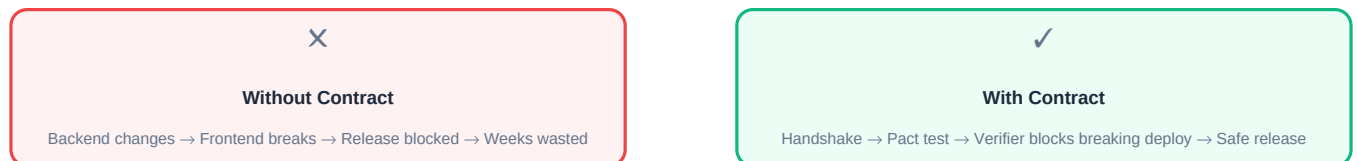
Why Contract Testing Matters

1.1 The Problem: Backend Breaks Frontend

Your frontend expects email but the backend ships emailAddress without warning. Your tests break. The release is blocked. You wait weeks for the backend to be ready before you can even start UI testing. This is not a people problem — it is a **contract problem**. No one wrote down what the API should look like, so no one noticed when it changed.

Contract testing is like a **handshake agreement before building** — both sides sign what the API will look like, so there are no surprises. The consumer (your UI) says "I need GET /users/1 → { id, name, email }" and the provider (backend) agrees. If either side breaks the handshake, the contract test fails **before** it reaches production.

Before vs After — The Cost of No Contract



What is a Contract?

A **contract** is a written agreement between **consumer** (who uses the API, e.g., your UI) and **provider** (who builds it, e.g., backend) about what requests and responses look like. Like a restaurant menu: you order what is on the menu, kitchen makes what is on the menu. If the menu changes without telling you, your order fails. The contract IS the menu.

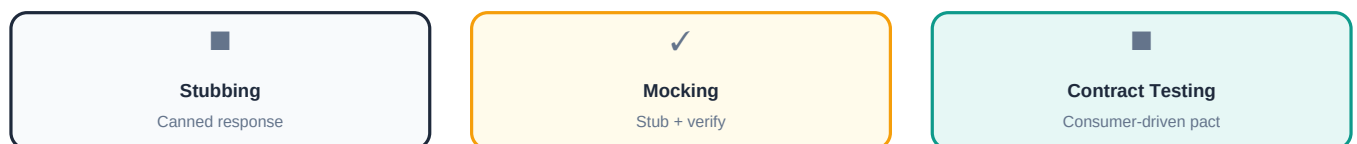
This kit gives you the menu (Pact file), the kitchen mock (WireMock), and the checklist that catches menu changes (breaking-change checklist) — so you never wait for the backend again.

1.2 Stub vs Mock vs Contract — Know the Difference

The #1 confusion for SDETs. Using a WireMock stub where a Pact contract is needed (or vice versa) gives false confidence. Here is the distinction, analogy-first:

- **Stub** — canned response for a matched request. Like a recorded voicemail — plays the same message regardless of who called. No verification that anyone called.
- **Mock** — stub + verification that expected requests were made. Like a voicemail that also tells you who called and when. WireMock does this via `verify(getRequestedFor(...))` or `GET /__admin/requests`.
- **Contract** — consumer-driven agreement + provider verification. Like a handshake + later inspection. Consumer writes `given / uponReceiving / willRespondWith`, Pact generates a JSON file, provider verifies via `Verifier`.

Three Concepts — Overlap Shows Tool Coverage



Concept	What It Is	Who Defines	Tool	When to Use
Stub	Canned response for matched request	Tester	WireMock stubFor	Need fixed data for UI dev
Mock	Stub + verification of calls	Tester	WireMock verify	Need to assert UI made correct calls
Contract	Consumer-driven pact + provider verify	Consumer	Pact V3 + Verifier	Prevent breaking changes in CI
WireMock	Does stub + mock	Tester	WireMock	UI dev + integration tests
Pact	Does contract	Consumer	Pact	CI gate, can-i-deploy

Rule of thumb: **WireMock for UI dev and integration tests** (what DOES happen), **Pact for CI contract gate** (what SHOULD happen). Together they cover the full loop — see p9.

1.3 Consumer-Driven vs Provider-Driven

Who owns the contract? In **consumer-driven** (Pact), the consumer writes the contract — "I need this shape" — and the provider must satisfy it. In **provider-driven** (OpenAPI), the provider publishes the spec — "I offer this shape" — and consumers validate against it.

Two Approaches — Who Defines the Contract?



Approach	Who Defines	Tool	When to Use	Pros / Cons
Consumer-Driven	Consumer	Pact	Consumer knows what it needs (most SDET cases)	Pros: precise. Cons: needs consumer tests
Provider-Driven	Provider	OpenAPI + validator	Provider has many consumers	Pros: one spec. Cons: consumers may not need all

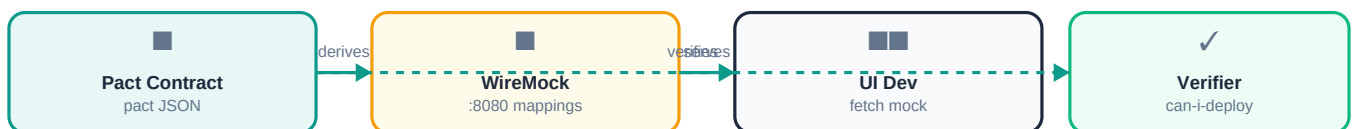
This Kit is Consumer-Driven (Pact)

As SDET you test what **YOUR UI needs**, not what the backend thinks you need. Consumer-driven means your Pact test defines the contract — if the backend breaks it, **Verifier fails and blocks deploy**. That is the power.

1.4 Pact + WireMock Together — Why Both?

Why not just Pact or just WireMock? Because they complement: **Pact = contract** (what SHOULD happen, the handshake), **WireMock = mock server** (what DOES happen during dev/test, the kitchen mock). Together they cover the full loop from UI dev to CI gate.

Pact → WireMock → UI → Verifier — The Full Loop



Tool	Role	When It Runs	Output
Pact	Contract — what SHOULD happen	Consumer test (Jest)	pacts/WebApp-UserAPI.json
WireMock	Mock server — what DOES happen	UI dev + E2E (Playwright)	http://localhost:8080 + mappings
Verifier	CI gate — blocks breaking deploy	Provider CI (can-i-deploy)	PASS / FAIL

WireMock Can Also RECORD

When the backend exists, WireMock can **record real API traffic** and generate mappings automatically:

--proxy-all="https://api.example.com" --record-mappings OR POST /__admin/recordings/start. Useful for capturing offline mocks. See p24.

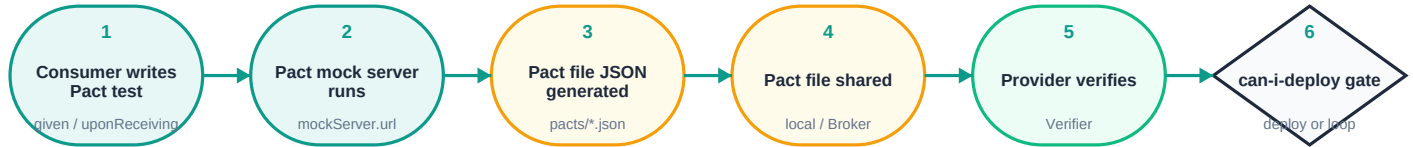
02

Lifecycle

Contract Testing Lifecycle

2.1 The 6-Step Consumer-Driven Loop

This is the mental model for all of Ch 3–6. Every Pact template and WireMock mapping fits into this loop.



- **1. Consumer writes Pact test** — `given('user exists') + uponReceiving + willRespondWith + executeTest`. Defines what the UI needs.
- **2. Pact mock server runs** — ephemeral, random port, `mockServer.url`. Your `fetch(mockServer.url + "/users/1")` hits it.
- **3. Pact file JSON generated** — `pacts/WebApp-UserAPI.json` with consumer, provider, interactions, metadata. This IS the contract.
- **4. Pact file shared** — local folder (this kit) or Pact Broker / PactFlow at scale (versioning + can-i-deploy).
- **5. Provider verifies** — Verifier with `providerBaseUrl + stateHandlers` seeds data, checks real provider satisfies pacts.
- **6. can-i-deploy gate** — checks if consumer and provider versions are compatible per pacts. Blocks deploy if verification failed. Loop back if fails.

Pact Broker is Optional

For this kit, we share via local `pacts/` folder. Pact Broker / PactFlow adds versioning + can-i-deploy at scale — useful when you have many consumers/providers, but not required to start.

2.2 Pact File Anatomy + can-i-deploy Gate

The PactV3 + `executeTest()` GENERATES a file — not just runs a test. And `can-i-deploy` is the CI gate that prevents breaking deploys.

Code → Artifact — Pact File Generation



Pact JSON structure (pretty-printed, truncated):

pacts/WebApp-UserAPI.json — the contract artifact

json

```
{
  "consumer": { "name": "WebApp" },
  "provider": { "name": "UserAPI" },
  "interactions": [
    {
      "description": "a request for user 1",
      "providerState": "user with id 1 exists",
      "request": { "method": "GET", "path": "/users/1" },
      "response": { "status": 200, "body": { "id": 1, "name": "Alice" } }
    }
  ],
  "metadata": { "pactSpecification": { "version": "3.0.0" } }
}
```

Field	What It Means
consumer.name	Who uses the API — your UI (WebApp)
provider.name	Who builds the API — backend (UserAPI)
interactions[]	Each request/response pair — one per Pact test
providerState	Data to seed before verification — given('user exists')
metadata.pactSpecification.version	Pact spec version — 3.0.0 for PactV3

The Pact File IS the Contract

Commit it, share it, version it. Without it, there is no contract testing. `can-i-deploy` checks if consumer and provider versions are compatible per pacts — blocks deploy if verification failed. That is the safety net.

03

Pact.js Templates

Five Ready-to-Use Tests — Promise 1

3.0 PactV3 Setup + Matchers Reference

What: This page installs Pact.js and explains matchers — the flexible rules that catch breaking changes.

What you need before starting: Node 18+ (node -v), npm, VS Code, terminal. See p3 for download links.

Where to do it: Terminal at companion_files/contract-mock-demo-app/ (or any folder with package.json).

How to do it: Copy-paste each command, one at a time:

Install Pact + Jest + Axios

bash

```
npm install --save-dev @pact-foundation/pact
# What: installs Pact 17.1.3 — the contract testing library
# Why: PactV3 + MatchersV3 for consumer-driven contracts
# Expected: "added 45 packages" + no errors

npm install --save-dev jest axios
# What: installs Jest 29 (test runner) + Axios (HTTP client)
# Why: Jest runs Pact tests, Axios makes requests to mock server
# Expected: "added 120 packages"
```

What you should see: package.json has "@pact-foundation/pact": "17.1.3", node_modules/@pact-foundation/pact exists, npx jest --version → 29.x.

What if it doesn't work? Pact install hangs → try PACT_DO_NOT_TRACK=1 npm install. Node version error → need 16+, see p3. Port conflict → Pact mock uses random port, no conflict.

What's next? Now you know matchers — next 5 pages each build one template (p13–p17).

Table T3 — Pact Matchers Reference (V3) — Cheat Sheet

Every matcher below is verified against Pact 17.1.3 + MatchersV3. Use this while writing templates.

Matcher	Example	What It Checks	When to Use
like	like("Alice")	Same type passes (any string)	Shape matters, value doesn't — most common
eachLike	eachLike("admin", {min:1})	Array with ≥1 item matching template	Lists — users, tags, orders
regex	regex('^[^@]+@[^@]+\.[^@]+\$', 'a@b.com')	String matching regex	Format matters — email, phone, UUID
integer	integer(1)	Any integer	IDs, counts, pagination
string	string("Alice")	Any string	Names, descriptions
boolean	boolean(true)	true or false	Flags — active, verified
timestamp	timestamp("yyyy-MM-dd'T'HH:mm:ss.SSSX")	String matching timestamp format	Dates — createdAt, updatedAt
atLeastLike	atLeastLike("x", 2)	Array with ≥N items	Lists with min size
constrainedArrayLike	constrainedArrayLike("x", 1, 5)	Array with min/max size	Bounded lists
arrayContaining	arrayContaining(["a", "b"])	Array containing values (any order)	Subset checks
fromProviderState	fromProviderState("\${id}", 1)	Value from provider state	Dynamic IDs from given
nullValue	nullValue()	null	Nullable fields
includes	includes("not found")	String containing substring	Error messages

V2 vs V3

This kit uses **PactV3 + MatchersV3** (current, stable since v10). V2's Pact + MatchersV2 + term still works — term({ matcher: 'F|M', generate: 'M' }) → V3 regex('F|M', 'M'). Both work, but V3 is recommended.

3.1 Template 01: GET /users/:id — Path Param + Type Matchers

What: Test GET /users/:id — verifies provider returns user with correct types. Simplest template, teaches PactV3 + like/integer/string + given/uponReceiving/willRespondWith/executeTest lifecycle.

What you need: Pact installed (p12), file at `companion_files/pact-templates/template-01-consumer-contract-basic.test.js` (or create).

Where: `companion_files/pact-templates/template-01-consumer-contract-basic.test.js` — copy to your project, or run in `companion_files/contract-mock-demo-app/tests/` for demo.

How: Full code — copy-paste ready, 40 lines, every matcher explained inline:

template-01-consumer-contract-basic.test.js — GET /users/:id

javascript

```
const path = require('path');
const { PactV3, MatchersV3 } = require('@pact-foundation/pact');
const axios = require('axios');
const { like, integer, string, regex, boolean, timestamp, eachLike } = MatchersV3;

const provider = new PactV3({
  consumer: 'WebApp',
  provider: 'UserAPI',
  dir: path.resolve(process.cwd(), 'pacts'),
});

describe('GET /users/:id', () => {
  it('returns user by id', async () => {
    provider
      .given('user with id 1 exists') // provider state – seed DB
      .uponReceiving('a request for user 1') // description
      .withRequest({
        method: 'GET',
        path: '/users/1',
        headers: { Accept: 'application/json' },
      })
      .willRespondWith({
        status: 200,
        headers: { 'Content-Type': 'application/json' },
        body: {
          id: integer(1), // any number – catches type drift
          name: string('Alice Johnson'), // any string – catches missing field
          email: regex('^[^@]+@[^@]+\.[^@]+$', 'alice@example.com'),
          active: boolean(true),
          createdAt: timestamp('yyyy-MM-dd\'T\'HH:mm:ss.SSSX', '2024-01-15T10:30:00.000Z'),
          tags: eachLike('admin', 1), // array ≥1 – catches shape drift
        },
      });

    await provider.executeTest(async (mockServer) => {
      const res = await axios.get(`${mockServer.url}/users/1`, {
        headers: { Accept: 'application/json' },
      });
      expect(res.status).toBe(200);
      expect(res.data.id).toBe(1);
    });
  });
});
```

What you should see: `npm test` → PASS `template-01` + `pacts/WebApp-UserAPI.json` generated with 1 interaction. `cat pacts/WebApp-UserAPI.json | head -20` shows consumer/provider/interactions.

What if it doesn't work? Pact file not generated → check `dir: path.resolve(process.cwd(), 'pacts')` exists, run `mkdir -p pacts`. Port conflict → Pact mock uses random port, no conflict.

What's next? Next: POST with body + regex (p14) — teaches regex and eachLike for format and array drift.

3.2 Template 02: POST /users — Body + Regex + Array Matchers

What: Test POST /users — verifies provider creates user and validates body matchers. Teaches regex for email format and eachLike for arrays.

What you need: Template 01 understood, same setup.

Where: companion_files/pact-templates/template-02-consumer-contract-create.test.js

How: Full code — POST with JSON body, regex catches format drift, eachLike catches array shape drift:

template-02 — POST /users (excerpt, full file has PactV3 setup)

javascript

```
provider
  .given('user creation is allowed')
  .uponReceiving('a request to create user Bob')
  .withRequest({
    method: 'POST',
    path: '/users',
    headers: { 'Content-Type': 'application/json' },
    body: {
      name: string('Bob'),
      email: regex('^[^@]+@[^@]+\.[^@]+$', 'bob@example.com'),
      tags: eachLike('admin', { min: 1 }),
    },
  })
  .willRespondWith({
    status: 201,
    headers: { 'Content-Type': 'application/json' },
    body: {
      id: integer(2),
      name: string('Bob'),
      email: regex('^[^@]+@[^@]+\.[^@]+$', 'bob@example.com'),
    },
  });

await provider.executeTest(async (mockServer) => {
  const res = await axios.post(`${mockServer.url}/users`, {
    name: 'Bob', email: 'bob@example.com', tags: ['admin'],
  });
  expect(res.status).toBe(201);
  expect(res.data.id).toBe(2);
});
```

Why regex?

Without regex, like('bob@example.com') would pass even if provider returns not-an-email. regex enforces format — if provider changes email format, test fails and blocks deploy. That is the value.

What you should see: PASS + pact file now has 2 interactions (GET + POST).

What if? 201 vs 200 → provider must return 201 for creation, check willRespondWith.status. Regex fails → test email must match pattern, see T3.

What's next? Next: GET with query params + fromProviderState (p15).

3.3 Template 03: GET /users?search=&page;= — Query Params + fromProviderState

What: Test GET /users?search=alice&page;=1 — verifies query param handling and dynamic data via provider states. Needed for search/pagination APIs.

What you need: Templates 01–02.

Where: companion_files/pact-templates/template-03-consumer-contract-search.test.js

How: Full code — query matching + provider states for dynamic IDs:

template-03 — GET /users?search (excerpt)

javascript

```
provider
  .given('users matching alice exist')
  .uponReceiving('a search for alice page 1')
  .withRequest({
    method: 'GET',
    path: '/users',
    query: { search: 'alice', page: '1' },
  })
  .willRespondWith({
    status: 200,
    headers: { 'Content-Type': 'application/json' },
    body: {
      users: eachLike({ id: integer(1), name: string('Alice Johnson') }),
      total: integer(10),
      page: integer(1),
      pageSize: integer(10),
    },
  });

// fromProviderState for dynamic IDs:
provider
  .given('user with id ${id} exists', { id: '42' })
  .uponReceiving('a request for user 42')
  .withRequest({ method: 'GET', path: '/users/42' })
  .willRespondWith({
    status: 200,
    body: { id: fromProviderState('${id}', 42), name: string('Alice') },
  });
```

Provider States

given('users matching alice exist') tells provider to seed data before verification. Handler is in `template-05-provider-verification.test.js` → `stateHandlers: { 'users matching alice exist': () => seedDB() }`. Without it, Verifier has no data.

What you should see: PASS + pact file 3 interactions.

What if? Query param order → Pact matches regardless of order. `fromProviderState` not working → need provider state handler in Verifier (see p17).

What's next? Next: error cases 404/422 (p16).

3.4 Template 04: Error Handling — 404 / 422 + nullValue + includes

What: Test error responses — verifies provider returns correct status + error body for failures. Without error contracts, provider can change error format and break UI error handling.

What you need: Templates 01–03.

Where: `companion_files/pact-templates/template-04-consumer-contract-error-handling.test.js`

How: Two tests in one file — 404 for unknown user, 422 for validation error:

template-04 — Error Handling (excerpt, 2 tests + nullValue)

javascript

```
// Test 1: 404 for unknown user
provider
  .given('user with id 999 does not exist')
  .uponReceiving('a request for non-existent user')
  .withRequest({ method: 'GET', path: '/users/999' })
  .willRespondWith({
    status: 404,
    headers: { 'Content-Type': 'application/json' },
    body: { error: includes('not found'), code: integer(404) },
  });

// Test 2: 422 for invalid body
provider
  .given('user creation with invalid email')
  .uponReceiving('a request to create user with bad email')
  .withRequest({
    method: 'POST', path: '/users',
    body: { name: string('Bob'), email: string('not-an-email') },
  })
  .willRespondWith({
    status: 422,
    body: {
      errors: eachLike({ field: string('email'), message: string('invalid email') }),
      code: integer(422),
    },
  });

// Nullable field example:
provider
  .given('user with nullable phone')
  .uponReceiving('a request for user with null phone')
  .withRequest({ method: 'GET', path: '/users/1' })
  .willRespondWith({
    status: 200,
    body: { id: integer(1), phone: nullValue(), name: string('Alice') },
  });
```

Error Contracts are Breaking-Change Detectors

If provider changes `{ error: 'msg' }` to `{ errors: [...] }`, this test fails and blocks deploy. That is the safety net for UI error handling — see checklist p30.

What you should see: PASS + pact file 5 interactions (2 error).

What if? 404 vs 200 → `willRespondWith.status` must match provider's actual error status. Error format change → this is breaking, see checklist p30.

What's next? Next: provider verification — the other half of contract testing (p17).

3.5 Template 05: Provider Verification — Verifier + stateHandlers

What: Verify provider against all 4 consumer pacts — the CI gate that catches breaking changes. Without this, contracts are just consumer wishes.

What you need: Templates 01–04 generated `pacts/WebApp-UserAPI.json`, provider running at `http://localhost:3000` (demo-app `server.js` or real API).

Where: `companion_files/pact-templates/template-05-provider-verification.test.js` + `companion_files/contract-mock-demo-app/tests/`

How: Full code — Verifier with `stateHandlers`, `pactUrls`, `providerBaseUrl`:

template-05-provider-verification.test.js — Verifier

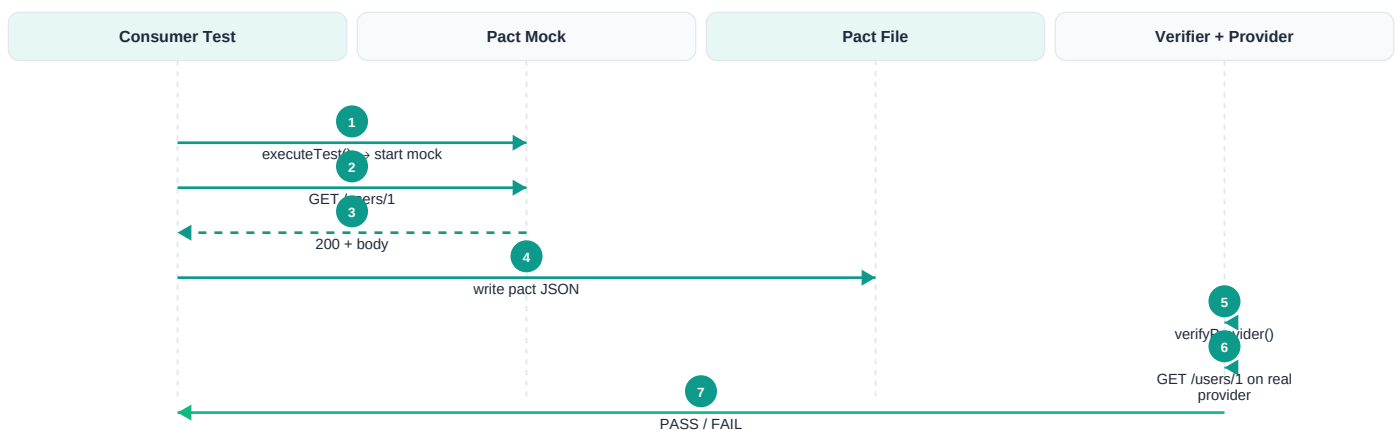
javascript

```

const { Verifier } = require('@pact-foundation/pact');
const path = require('path');

describe('Pact Verification', () => {
  it('validates provider against consumer pacts', async () => {
    await new Verifier({
      provider: 'UserAPI',
      providerBaseUrl: 'http://localhost:3000',
      pactUrls: [path.resolve(process.cwd(), 'pacts/WebApp-UserAPI.json')],
      // OR pactBrokerUrl: 'https://my-broker.example.com',
      stateHandlers: {
        'user with id 1 exists': () => {
          // seed DB / stub dependencies
          return Promise.resolve({ description: 'user seeded' });
        },
        'users matching alice exist': () => {
          return Promise.resolve({ description: 'search data seeded' });
        },
        'user with id 999 does not exist': () => Promise.resolve(),
      },
      // requestFilter: (req, res, next) => {
      //   req.headers['Authorization'] = 'Bearer token'; next();
      // },
    }).verifyProvider().then(() => console.log('Pact Verification Complete!'));
  });
});

```



What you should see: Pact Verification Complete! + all interactions PASS. If provider breaks contract → FAIL with diff (expected vs actual).

What if? Verifier fails → check provider is running (`curl http://localhost:3000/health`), check stateHandlers seed correctly. Pact file not found → check dir in consumer tests.

What's next? Next: run all 5 together + see the pact file (p18).

3.6 Running All 5 + Pact File Output

What: Run all 5 templates and inspect the pact file — your contract artifact.

What you need: All 5 templates, provider running for 05.

Where: Terminal at `companion_files/contract-mock-demo-app/` or any project with pacts/.

How: One command runs all 5, then inspect the JSON:

```
Run all 5 templates
npm test
# What: runs Jest, all 5 files (01-04 consumer + 05 verifier)
# Why: generates pacts/ + verifies provider
# Expected: 5 PASS + 1 verification PASS

# Inspect the pact file:
cat pacts/WebApp-UserAPI.json | head -50
# What: shows pact JSON structure
# Expected: consumer, provider, interactions array, metadata
```

Expected terminal output:

```
Terminal — npm test output (expected)
PASS template-01-consumer-contract-basic.test.js
PASS template-02-consumer-contract-create.test.js
PASS template-03-consumer-contract-search.test.js
PASS template-04-consumer-contract-error-handling.test.js
PASS template-05-provider-verification.test.js

Pact Verification Complete!
Test Suites: 5 passed, 5 total
Tests:      6 passed, 6 total

$ ls pacts/
WebApp-UserAPI.json
```

What you should see: 5 tests PASS, `pacts/WebApp-UserAPI.json` exists, `ls pacts/` shows file. Pact file has 10 interactions (GET, POST, search, 2 errors, etc.).

What if? One test fails → read diff, fix matcher or provider. No `pacts/` folder → `mkdir -p pacts` and re-run.

What's next? You now have contracts — next: mock server to test UI while backend builds (Ch 4, p20).

3.7 Common Pitfalls + Matcher Cheat Sheet

Prevent common mistakes that waste hours:

Pitfall	Symptom	Fix
Using Pact instead of PactV3	Deprecation warning	Use PactV3 + MatchersV3
Pact file not generated	No dir or not calling <code>executeTest</code>	Add <code>dir: path.resolve(...)</code> + use <code>executeTest</code>
Matcher too loose (<code>like</code>)	Misses format drift	Use <code>regex</code> for formats (email, UUID)
Matcher too strict (<code>equal</code>)	Brittle — fails on any value change	Use <code>like</code> for types
Forgetting given	Provider state not seeded	Add <code>given</code> + <code>stateHandler</code>

Rule of Thumb

like for shape, **regex** for format, **eachLike** for lists. Start loose, tighten where format matters. `integer/string` for primitives, `fromProviderState` for dynamic IDs. See T3 (p12) for full reference.

04

WireMock

Three Setup Scripts — Promise 2

4.1 Setup Options Comparison — Choose Your Path

Pick JAR vs Docker vs Compose in 30 seconds. Wrong choice wastes setup time.

Option	Command	Needs Java?	Best For	Pros / Cons
Standalone JAR	<code>java -jar wiremock-standalone-3.13.2.jar --port 8080 --verbose</code>	Yes (11+)	No Docker, just Java	Pros: direct. Cons: needs Java 11+
Docker run	<code>docker run -p 8080:8080 wiremock/wiremock:3.13.2 --verbose</code>	No	Has Docker, no Java	Pros: no Java. Cons: needs Docker
Docker Compose	<code>docker-compose up</code>	No	Team sharing, one command	Pros: one command. Cons: needs compose

All 3 Produce the Same Mock Server

All three produce the same mock server at **http://localhost:8080** — pick one, the mappings are identical. Mappings are in `companion_files/wiremock-setup/03-mappings/mappings/` (3 JSON files) + `__files/`.

4.2 Script 01: Standalone JAR — java -jar

What: Start WireMock via standalone JAR — no Docker needed, just Java 11+.

What you need: Java 11+ (java -version → 17 recommended, adoptium.net), wiremock-standalone-3.13.2.jar downloaded, wiremock/mappings/ + __files/ folders.

Where: Terminal at `companion_files/wiremock-setup/`

How: One command — each flag explained:

Script 01 — Standalone JAR (Mac/Linux/Windows — Java is cross-platform)

bash

```
java -jar wiremock-standalone-3.13.2.jar \
  --port 8080 \
  --global-response-templating \
  --enable-stub-cors \
  --root-dir ./wiremock \
  --verbose
# --port: mock server port (change if 8080 in use)
# --global-response-templating: enables Handlebars for all stubs
# --enable-stub-cors: adds Access-Control-Allow-Origin: * (needed for UI)
# --root-dir: folder with mappings/ + __files/ subfolders
# --verbose: logs every request + matching (debugging)
```

Verify — curl the mock server

bash

```
curl http://localhost:8080/__admin/mappings
# Expected: {"mappings": [3 mappings]}

curl http://localhost:8080/users/1
# Expected: {"id":1,"name":"Alice Johnson",...}
```

What you should see: WireMock started on port 8080 + `curl http://localhost:8080/__admin/mappings` → `{"mappings": [...]}` (3 mappings).

What if? Java not found → install Temurin 17 from adoptium.net. Port 8080 in use → use `--port 9999` and update `{{baseUrl}}`. JAR not found → check download path (repo1.maven.org).

What's next? Next: Docker option (p22) — or skip to mappings (p23) if JAR works.

4.3 Script 02: Docker + Docker Compose — No Java Needed

What: Start WireMock via Docker — no Java, one command, same mock server.

What you need: Docker Desktop (docker --version, [docker.com](https://docs.docker.com)), wiremock/mappings/ + __files/ folders.

Where: Terminal at companion_files/wiremock-setup/ (has docker-compose.yml).

How: Option A — docker run, Option B — docker-compose up:

Script 02 — Docker run + Compose

bash

```
docker run -it --rm -p 8080:8080 \
  -v $PWD/wiremock:/home/wiremock \
  --name wiremock wiremock/wiremock:3.13.2 \
  --global-response-templating --enable-stub-cors --verbose
# -p: port mapping 8080:8080
# -v: mount local wiremock/ to /home/wiremock in container
# --name: container name
# Image: wiremock/wiremock:3.13.2 (official since 2.31.0)

# Windows PowerShell variant:
# -v ${PWD}/wiremock:/home/wiremock

# Option B — Compose (one command):
docker-compose up
# Or: docker compose up (newer Docker)
```

docker-compose.yml (in companion_files/wiremock-setup/)

yaml

```
services:
  wiremock:
    image: "wiremock/wiremock:3.13.2"
    ports: ["8080:8080"]
    volumes: [".:/wiremock:/home/wiremock"]
    command: ["--global-response-templating", "--enable-stub-cors", "--verbose"]
```

Table T4 — WireMock CLI Flags (Essential)

Flag	Example	What It Does	When to Use
--port	--port 8080	Sets mock server port	Always — change if 8080 in use
--root-dir	--root-dir ./wiremock	Folder with mappings/ + __files/	Always — points to your mappings
--global-response-templating	--global-response-templating	Enables Handlebars for all stubs	Needed for dynamic mocks (p24)
--enable-stub-cors	--enable-stub-cors	Adds Access-Control-Allow-Origin: *	Always for UI-against-mock (p27)
--verbose	--verbose	Logs every request + matching	Debugging — see why stub didn't match
--https-port	--https-port 8443	Enables HTTPS	If UI uses https
--record-mappings	--record-mappings	Auto-records proxied requests	Capture real API traffic (p24)
--proxy-all	--proxy-all="https://api.example.com"	Proxies unmatched to real API	Partial backend + record mode
--disable-gzip	--disable-gzip	Disables gzip	If client doesn't handle gzip

What you should see: WireMock started + curl http://localhost:8080/__admin/mappings same as JAR (3 mappings).

What if? Docker not found → install Docker Desktop. Volume mount fails on Windows → use \${PWD} in PowerShell, %cd% in cmd. Port conflict → change 8080:8080 to 9999:8080.

What's next? Next: mappings — the stubs that define mock responses (p23).

4.4 Script 03: Mappings + __files — Define Mock Responses

What: Create 3 WireMock mappings — the stubs that return mock data for GET/POST/search. Without these, WireMock returns 404 for every request.

What you need: WireMock running (p21 or p22), wiremock/mappings/ folder.

Where: companion_files/wiremock-setup/03-mappings/mappings/ (3 files) + __files/ (body files).

How: Each mapping is a JSON file — request matcher + response. Three examples:

03-mappings/mappings/get-user.json — GET /users/1

json

```
{
  "request": {
    "method": "GET",
    "url": "/users/1",
    "headers": { "Accept": { "equalTo": "application/json" } }
  },
  "response": {
    "status": 200,
    "headers": {
      "Content-Type": "application/json",
      "Access-Control-Allow-Origin": "*"
    },
    "jsonBody": {
      "id": 1, "name": "Alice Johnson",
      "email": "alice@example.com", "active": true
    }
  }
}
```

create-user.json — POST /users (201)

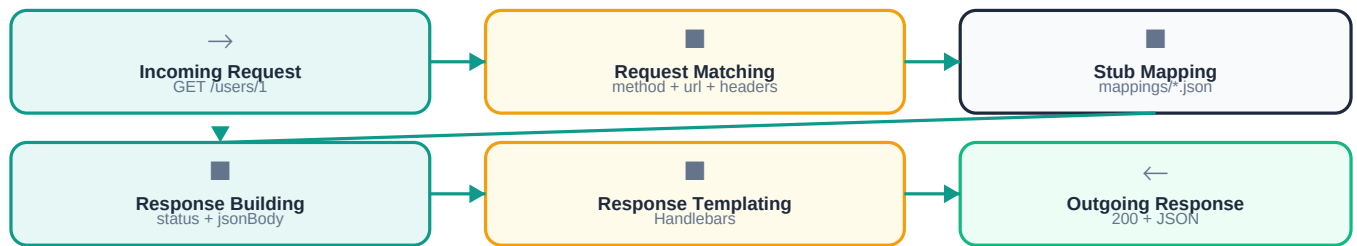
json

```
{
  "request": {
    "method": "POST", "url": "/users",
    "headers": { "Content-Type": { "contains": "json" } },
    "bodyPatterns": [{ "matchesJsonPath": "$.name" }]
  },
  "response": {
    "status": 201,
    "headers": { "Content-Type": "application/json" },
    "jsonBody": { "id": 2, "name": "Bob", "email": "bob@example.com" }
  }
}
```

search-users.json — GET /users?search=alice&page=1

json

```
{
  "request": {
    "method": "GET",
    "urlPath": "/users",
    "queryParameters": {
      "search": { "equalTo": "alice" },
      "page": { "equalTo": "1" }
    }
  },
  "response": {
    "status": 200,
    "jsonBody": {
      "users": [{ "id": 1, "name": "Alice Johnson" }],
      "total": 1, "page": 1, "pageSize": 10
    }
  }
}
```

WireMock Pipeline — Request → Matching → Response

What you should see: `curl http://localhost:8080/users/1 → 200 + JSON`, `curl http://localhost:8080/__admin/mappings` shows 3 mappings.

What if? 404 → check mapping file is valid JSON, check WireMock logs `--verbose`. CORS blocked → ensure mapping has `Access-Control-Allow-Origin: *`. Mapping not loaded → check `--root-dir` points to folder with `mappings/` subfolder.

What's next? Next: templating + record/playback — make mocks dynamic (p24).

4.5 Response Templating + Record/Playback — Dynamic Mocks

What: Make mocks dynamic with Handlebars templating and capture real API traffic via record/playback.

What you need: WireMock running with `--global-response-templating`, mappings with transformers.

Where: `wiremock/mappings/` (add templated mapping) + `__admin` API.

How: Templating example — echo request data, random values, timestamps:

Templated mapping — Handlebars (needs transformers or `--global-response-templating`)

json

```
{
  "request": { "urlPath": "/users/1" },
  "response": {
    "status": 200,
    "body": "{ \"id\": \"{{request.pathSegments.[1]}}\", \"timestamp\": \"{{now format='yyyy-MM-dd'}}\" }",
    "headers": { "Content-Type": "application/json" },
    "transformers": ["response-template"]
  }
}
# Helpers: {{request.path}}, {{request.query.search}}, {{now}}, {{randomValue type='UUID'}}, {{jsonPath request.
body '$.name'}}
```

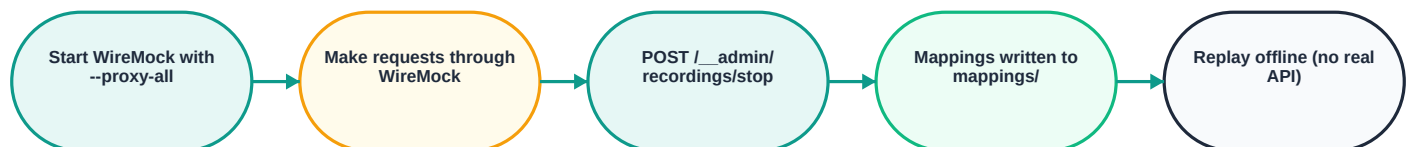
Record/playback — capture real API traffic:

Record / Playback — capture real API

bash

```
# Start WireMock as proxy + recorder:
java -jar wiremock-standalone-3.13.2.jar \
  --port 8080 --proxy-all="https://api.example.com" --record-mappings --verbose

# Or via Admin API (WireMock already running):
curl -X POST http://localhost:8080/__admin/recordings/start \
  -d '{ "targetBaseUrl": "https://api.example.com" }'
# Make requests through WireMock (they proxy to real API + get recorded)
curl http://localhost:8080/users/1
# Stop recording — mappings written to mappings/:
curl -X POST http://localhost:8080/__admin/recordings/stop
# Or snapshot:
curl -X POST http://localhost:8080/__admin/recordings/snapshot
ls wiremock/mappings/ # shows recorded files
```



What you should see: `curl http://localhost:8080/users/1` returns templated response with dynamic timestamp, `ls wiremock/mappings/` shows recorded files.

What if? Templating not working → need transformers: `["response-template"]` or `--global-response-templating`. Record not capturing → check `targetBaseUrl` is reachable, check `--proxy-all`.

What's next? Mocks ready — next: validate them with Postman (Ch 5, p25).

05

Postman

Collection — Validate Mock Responses — Promise 4

5.1 Collection Structure + Variables — Import and Configure

What: Import the Postman collection and configure it to hit your WireMock mock server.

What you need: Postman installed (postman.com/downloads), companion_files/postman/Contract_Mock_Validation.postman_collection.json (v2.1), WireMock running at http://localhost:8080.

Where: Postman → File → Import → select JSON; then collection Variables.

How: Import steps + configure {{baseUrl}}:

Import + Configure — step by step

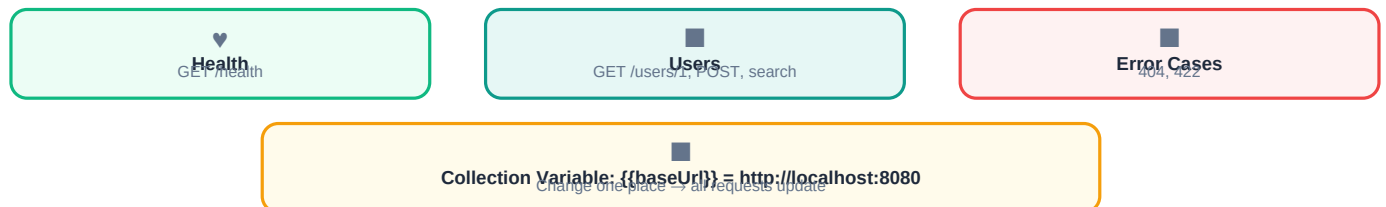
bash

1. Open Postman → File → Import → Upload Files
2. Select: companion_files/postman/Contract_Mock_Validation.postman_collection.json
3. Click Import → collection appears in sidebar
4. Click collection → Variables tab → set baseUrl = http://localhost:8080
5. Save (Ctrl+S)

Collection structure (3 folders, 6 requests):

```
Health/  
  GET {{baseUrl}}/health  
Users/  
  GET {{baseUrl}}/users/1  
  POST {{baseUrl}}/users  
  GET {{baseUrl}}/users?search=alice&page=1  
Error Cases/  
  GET {{baseUrl}}/users/999 → 404  
  POST {{baseUrl}}/users (invalid) → 422
```

Collection Structure — 3 Folders + baseUrl Variable



What you should see: Collection appears in Postman sidebar with 3 folders, 6 requests, variable baseUrl set to http://localhost:8080.

What if? Import fails → validate JSON at schema.getpostman.com. Variable not set → check collection Variables tab, not Environment.

What's next? Next: run requests and see pm.test assertions validate responses (p26).

5.2 pm.test Assertions + Validation Workflow — Run and Verify

What: Run Postman requests against WireMock and see assertions validate every response.

What you need: Collection imported (p25), WireMock running with mappings (p23).

Where: Postman → click request → Send → see Tests tab (Post-response).

How: Click GET {{baseUrl}}/users/1 → Send → see Tests results. Each request has pm.test + pm.expect (Chai BDD):

pm.test examples — Post-response tab (not Pre-request)

javascript

```
pm.test("Status is 200", () => {
  pm.response.to.have.status(200);
});

pm.test("Body has correct fields", () => {
  const j = pm.response.json();
  pm.expect(j.id).to.be.a('number');
  pm.expect(j.name).to.eql('Alice Johnson');
  pm.expect(j.email).to.include('@');
});

pm.test("Response time <500ms", () => {
  pm.expect(pm.response.responseTime).to.be.below(500);
});

pm.test("Schema valid", () => {
  pm.response.to.have.jsonSchema(schema);
});

// Chaining — save userId for next request:
pm.collectionVariables.set("userId", pm.response.json().id);
// Next request uses: GET {{baseUrl}}/users/{{userId}}
```

Table T6 — pm.test Assertions Reference — Cheat Sheet

Assertion	Code	What It Validates
Status	<code>pm.response.to.have.status(200)</code>	Status code is 200
Header	<code>pm.response.to.have.header("Content-Type")</code>	Header exists and matches
JSON body	<code>pm.expect(j.id).to.be.a('number')</code>	Body field type and value
JSON schema	<code>pm.response.to.have.jsonSchema(schema)</code>	Body matches JSON schema (tv4/ajv)
Response time	<code>pm.expect(pm.response.responseTime).to.be.below(500)</code>	Response time <500ms
Collection variable	<code>pm.collectionVariables.set("userId", j.id)</code>	Save value for next request
Chaining	<code>GET {{baseUrl}}/users/{{userId}}</code>	Use saved variable in next request URL

What you should see: All tests green (PASS), response body matches WireMock mapping, response time <500ms. Collection-level event checks response time + content-type for every request.

What if? Tests fail → check WireMock is running, check `baseUrl` variable, check mapping returns expected status. `pm.test` not running → check Scripts > Post-response tab (not Pre-request).

What's next? Mocks validated — next: the hero workflow — test UI against mock while backend builds (Ch 6, p27).

06

UI Against Mock

Exact Setup While Backend Builds — HERO — Promise 5

6.1 End-to-End Sequence — The Full Workflow

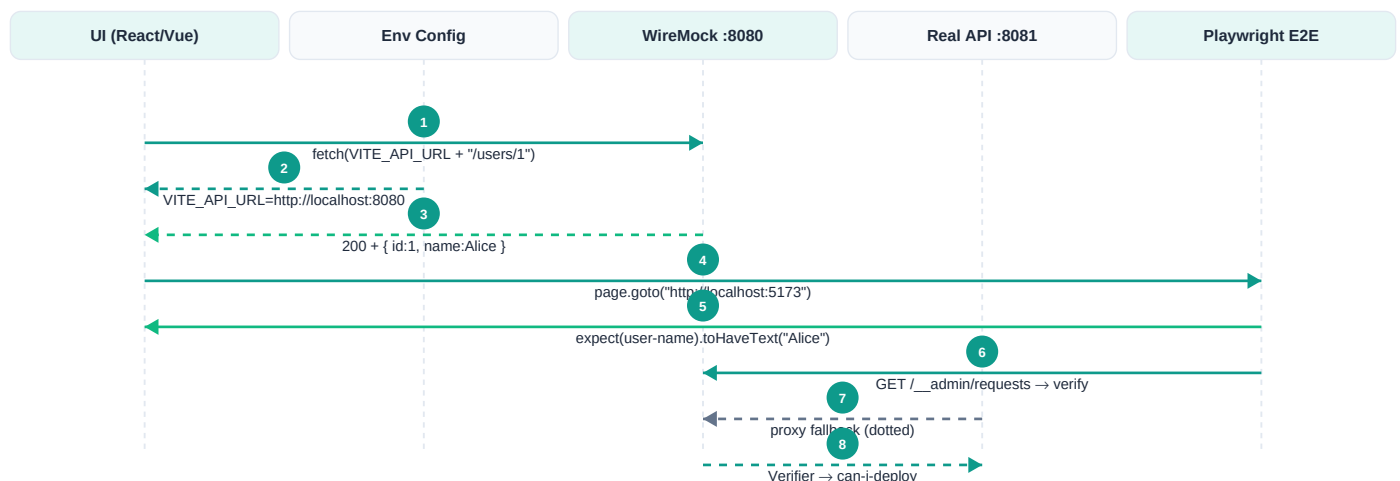
What: Set up the full loop: Pact contract → WireMock mappings → UI points to mock → E2E tests pass — all before backend exists. This is the most differentiated value in the kit.

What you need: Pact templates (Ch 3) generated pact JSON, WireMock running (Ch 4), UI code (`index.html` or any React/Vue app with `fetch`), Playwright or Cypress (optional).

Where: Terminal + VS Code + browser. Env var at `.env`, WireMock at `:8080`, UI at `:5173` OR `:3000`.

How: 10 steps — each with command/file:

1. Write Pact consumer test (Ch 3) → `given/uponReceiving/willRespondWith`
2. Generate pact JSON → `pacts/WebApp-UserAPI.json` via `executeTest`
3. Create WireMock mappings from pact (or hand-write to match, see p23)
4. Start WireMock → `docker run wiremock:3.13.2 --global-response-templating --enable-stub-cors`
5. Set UI env var → `VITE_API_URL=http://localhost:8080` (see p28 for framework table)
6. Run UI → `npm run dev` → UI at `http://localhost:5173`
7. UI calls `fetch(VITE_API_URL + "/users/1")` → hits WireMock → returns mock JSON → UI renders
8. Run Playwright → `npx playwright test` against UI
9. Verify via GET `http://localhost:8080/__admin/requests` → shows UI's requests
10. Later: real API ready → Verifier checks pact → switch env to real URL → `can-i-deploy`



HERO DIAGRAM — 10-step UI-against-mock sequence. Teal = mock path, slate dashed = real API (future), red callout = CORS. This is the exact setup promised.

What you should see: UI shows mock user data (Alice), Playwright PASS, WireMock `/__admin/requests` shows UI's GET `/users/1`.

What if? UI not hitting mock → check env var, restart dev server, check browser Network tab. CORS blocked → see p28.

What's next? Next: env var + CORS + proxy details (p28).

6.2 Env Var Switch + CORS + Proxy Fallback — The Details That Make It Work

What: Configure the three details that make UI-against-mock work: env var, CORS, proxy. Miss any and the workflow breaks.

What you need: UI project with env support, WireMock running.

Where: .env file + WireMock mappings + WireMock proxy config.

How: Three details, each with fix:

1. Env var switch — no code change, just config:

Framework	Env Var	Usage in Code
Vite (React/Vue)	VITE_API_URL	fetch(import.meta.env.VITE_API_URL + "/users/1")
Create React App	REACT_APP_API_URL	fetch(process.env.REACT_APP_API_URL + "/users/1")
Next.js	NEXT_PUBLIC_API_URL	fetch(process.env.NEXT_PUBLIC_API_URL + "/users/1")
Plain HTML	window.API_URL	fetch(window.API_URL + "/users/1")

```
.env — one line switches mock ↔ real
VITE_API_URL=http://localhost:8080
# In .env — change one line to switch mock ↔ real:
# Mock: VITE_API_URL=http://localhost:8080
# Real: VITE_API_URL=http://localhost:3000 (or https://api.example.com)
```

bash

2. CORS — browser blocks without it:

WireMock must send Access-Control-Allow-Origin: * — via flag OR mapping header:

```
CORS — two ways to enable
# Via flag (recommended):
java -jar wiremock-standalone-3.13.2.jar --enable-stub-cors
# Or docker: docker run ... --enable-stub-cors

# Via mapping header:
{
  "response": {
    "headers": { "Access-Control-Allow-Origin": "*" }
  }
}
```

json

3. Proxy fallback — partial backend: If some endpoints are ready on real backend, proxy unmatched requests:

```
Proxy fallback — unmatched → real API
{
  "request": { "urlPattern": "/api/.*" },
  "response": { "proxyBaseUrl": "https://real-api.example.com" }
}
# Unmatched requests proxy through to real API — matched ones return mocks.
// WireMock → Real API for unmatched (dotted line in D5)
```

json

What you should see: Browser Network tab shows 200 from localhost:8080, no CORS error, UI renders Alice.

What if? CORS error in console → add --enable-stub-cors or header. Env var not picked up → restart dev server (Vite/CRA need restart).

Proxy not working → check proxyBaseUrl is reachable.

What's next? Next: E2E verification with Playwright (p29).

6.3 Playwright E2E Against WireMock + Verification — Prove It Works

What: Run Playwright E2E tests against UI (which hits WireMock) and verify WireMock received correct requests — the proof that UI-against-mock is testable, not just manual.

What you need: UI running at http://localhost:5173 (or 3000), WireMock at 8080, Playwright installed (npm install -D @playwright/test).

Where: companion_files/contract-mock-demo-app/ or any UI project.

How: Playwright test + WireMock verification:

Playwright E2E — UI against WireMock

javascript

```
import { test, expect } from '@playwright/test';

test('UI shows user from mock', async ({ page }) => {
  await page.goto('http://localhost:5173');
  // data-testid is the resilient selector – not CSS class
  await expect(page.locator('[data-testid="user-name"]').toHaveText('Alice Johnson');
  await expect(page.locator('[data-testid="user-email"]').toContainText('alice@example.com'));
});

// Verify WireMock received the request:
test('WireMock received GET /users/1', async () => {
  const res = await fetch('http://localhost:8080/__admin/requests');
  const data = await res.json();
  const reqs = data.requests.map(r => r.request.url);
  expect(reqs).toContain('/users/1');
});
```

Run + Verify — terminal output (expected)

bash

```
npx playwright test
# Expected:
# ✓ UI shows user from mock (2.1s)
# ✓ WireMock received GET /users/1 (0.3s)
# 2 passed

curl http://localhost:8080/__admin/requests | head -20
# Expected: { "requests": [{ "request": { "method": "GET", "url": "/users/1" } } ] }
```

What you should see: Playwright PASS (2 tests), WireMock requests log shows GET /users/1, UI screenshot shows Alice.

What if? Playwright fails → check UI is running, check WireMock is running, check baseUrl. No requests in log → UI not hitting WireMock, check env var.

What's next? UI workflow complete — next: breaking-change checklist to prevent future breaks (Ch 7, p30).

07

Breaking-Change Checklist

Catch Schema Drift Early — Promise 3

7.1 How to Use the Checklist — Severity Legend + When to Run

Run this checklist for **every API change** — before merging PR, before deploying provider, in CI via `can-i-deploy`, when adding new endpoint.

Severity	Meaning	Pact Result	Action
■ Breaking	Must fix before deploy	Verifier FAILS	Block deploy, fix provider
■ Caution	Breaking if consumer checks that field/status	Verifier fails only if asserted	Check if consumer asserts it
■ Safe	Non-breaking — safe to deploy	Verifier PASSES	Deploy — no contract affected

When to run:

- Before merging any API PR — check every changed field/status/header
- Before deploying provider — run `can-i-deploy` in CI
- When adding new endpoint — is it optional (■) or required (■)?
- When changing error format — does UI error handling depend on old shape?

Print the Quick-Ref

Print the one-page quick-ref (p33) and pin it next to your desk. Run through it for every API change. Printable at `companion_files/checklists/api-breaking-change-checklist.md` + `companion_files/breaking-change-checklist/API_Breaking_Change_Checklist.md`.

7.2 Request Breaking Changes — URL, Method, Headers, Query, Body

Half the breaking changes are on the request side. Each item has severity, example, detection (which Pact matcher catches it), and fix.

☐	URL / Path changed /users/:id → /v2/users/:id — Pact path mismatch, no interaction matches	BREAKING
☐	HTTP method changed GET → POST — method mismatch, Pact fails	BREAKING
☐	Required header removed/renamed X-Request-Id removed — header assertion fails if consumer checks	CAUTION
☐	Query param changed page → pageSize — query assertion fails	CAUTION
☐	Required field added to POST body phone added as required — provider rejects without new field	BREAKING
☐	Auth scheme changed Bearer → ApiKey — 401, Pact auth header fails	BREAKING
☐	Pagination param changed page → cursor — pagination contract breaks	BREAKING

Change Type	Example	Severity	Pact Matcher	Fix
URL / Path	/users/:id → /v2/users/:id	■ Breaking	Path mismatch	Version API, keep old path
HTTP Method	GET → POST	■ Breaking	Method mismatch	Keep old method, add new endpoint
Required Header	X-Request-Id removed	■ Caution	Header assertion	Keep header, deprecate slowly
Query Param	page → pageSize	■ Caution	Query assertion	Support both, alias
Required Field (body)	Add phone (required)	■ Breaking	Body matcher fails	Make optional, add default
Auth Scheme	Bearer → ApiKey	■ Breaking	Auth header fails	Support both during migration
Pagination	page → cursor	■ Breaking	Pagination matcher	Version pagination

7.3 Response Breaking Changes — Status, Headers, Body, Type, Enum, Pagination, Errors

Other half — response changes that break UI rendering. These are often silent until production.

☐	Field removed email removed — like fails, consumer expects field	BREAKING
☐	Type changed id: string → number — integer vs string fails	BREAKING
☐	Enum narrowed status: active inactive pending → active inactive — regex fails	BREAKING
☐	Status code changed 200 → 204 — willRespondWith status fails	CAUTION
☐	Pagination changed { users: [...] } → { data: [...], meta } — JSON path fails	BREAKING
☐	Error format changed { error: "msg" } → { errors: [...] } — error matcher fails	CAUTION
☐	Content-Type changed application/json → vnd.api+json — header fails	CAUTION
☐	Optional field added nickname added — safe, like ignores extra fields	SAFE

Change Type	Example	Severity	Pact Matcher	Fix
Field Removed	email removed	■ Breaking	like fails	Keep field, deprecate
Type Changed	id: string → number	■ Breaking	integer vs string	Keep type, version

Change Type	Example	Severity	Pact Matcher	Fix
Enum Narrowed	status: 3 → 2 values	■ Breaking	regex fails	Keep enum, add new
Status Code	200 → 204	■ Caution	Status mismatch	Keep 200, add 204 as alt
Pagination	{ users } → { data, meta }	■ Breaking	JSON path fails	Version pagination
Error Format	{ error } → { errors }	■ Caution	Error matcher fails	Support both formats
Content-Type	json → vnd.api+json	■ Caution	Header fails	Keep json, add new type
Optional Field Added	nickname added	■ Safe	like ignores extra	Safe — deploy

7.4 Decision Tree + Printable Quick-Ref + Sign-Off

Fast decision tree — "is this change breaking?" — plus one-page quick-ref and sign-off table for team use.



Decision tree — each diamond is a yes/no. ■ = breaking (blocks deploy), ■ = caution (if consumer checks), ■ = safe. Each leaf shows which Pact matcher catches it.

One-Page Quick-Ref — Condensed T7 (14 rows, severity color-coded)

#	Change	Severity	Matcher	Fix
1	Field removed	■	like	Keep field
2	Required field added	■	body	Make optional
3	Type changed	■	integer/string	Keep type
4	Enum narrowed	■	regex	Keep enum
5	URL / Path changed	■	path	Version API
6	Method changed	■	method	Keep method
7	Auth changed	■	header	Support both
8	Pagination changed	■	jsonPath	Version
9	Status code changed	■	status	Keep 200
10	Header changed	■	header	Keep header
11	Query param changed	■	query	Support both
12	Error format changed	■	includes	Support both
13	Content-Type changed	■	header	Keep json
14	Optional field added	■	like (ignores)	Safe — deploy

Sign-Off Table — For Team Reviews

Date	Change Description	Severity	Checked By	Approved
2026-__-__	e.g., Remove email field from GET /users/:id	■		■
2026-__-__				■
2026-__-__				■

Printable

Printable checklist: `companion_files/checklists/api-breaking-change-checklist.md` + `companion_files/breaking-change-checklist/API_Breaking_Change_Checklist.md` — print and use for every API review. This 4-page section (p30–p33) + printable = 10-page equivalent when printed with detail.

App Troubleshooting

8 Common Issues — Quick Fixes

Every fix is copy-paste. Check **Where to Check** column for verification.

Symptom	Cause	Fix (copy-paste)	Where to Check
Pact file not generated	No dir or not calling executeTest	<code>mkdir -p pacts + check dir: path.resolve(...)</code>	<code>ls pacts/</code>
WireMock 404 for /users/1	Mapping not loaded / invalid JSON	Check JSON valid + <code>--root-dir</code> has mappings/subfolder	<code>curl http://localhost:8080/__admin/mappings</code>
CORS blocked in browser	Missing CORS header	Add <code>--enable-stub-cors</code> or header <code>Access-Control-Allow-Origin: *</code>	Browser Network tab → Headers
Postman variable not set	baseUrl not configured	Collection → Variables → baseUrl = <code>http://localhost:8080</code>	Postman Variables tab
Verifier fails — ECONNREFUSED	Provider not running	Terminal 1: <code>npm start</code> (port 3000) before Terminal 2: <code>npm test</code>	<code>curl http://localhost:3000/health</code>
Docker port conflict	8080 already in use	Change <code>-p 8080:8080</code> to <code>-p 9999:8080</code> + update <code>{{baseUrl}}</code>	<code>docker ps + lsof -i :8080</code>
Java version error	Java <11 for WireMock 3.x	Install Temurin 17: adoptium.net/temurin/releases/	<code>java -version</code> → 17.x
pm.test not running	Wrong tab — Pre-request vs Post-response	Move <code>pm.test</code> to Scripts → Post-response tab	Postman Scripts tab

Still Stuck?

Check `companion_files/contract-mock-demo-app/README.md` troubleshooting or open an issue with `npm --version + node -v + error log`.
Every tool is free/OSS — no vendor lock-in, no paid support needed.

Glossary — 20 Terms, Analogy-First, Jargon-Free

Every term defined on first use in the guide. This table is the reference.

Term	Plain-Language Definition (analogy-first)	Where It Fits
Consumer	The app that USES the API — like a customer ordering from a menu. Your UI/frontend.	Writes Pact contract
Provider	The app that BUILDS the API — like the kitchen making the dish. Backend API.	Must satisfy contract
Contract	Written agreement: what requests + responses look like. Like a menu.	Pact file (pacts/*.json)
Pact file	JSON artifact generated by consumer tests — the contract. Commit & share it.	pacts/WebApp-UserAPI.json
Provider state	given('user exists') — tells provider what data to seed before verification.	stateHandlers in Verifier
Matcher	Flexible rule: like (any type), regex (format), eachLike (array). Catches drift.	willRespondWith body
Verifier	Checks real provider satisfies all consumer pacts. CI gate.	template-05, can-i-deploy
Stub	Canned response for a matched request — no verification. Like voicemail.	WireMock mappings/*.json
Mock	Stub + verification that expected requests were made. Voicemail + caller log.	WireMock verify / __admin/requests
WireMock mapping	JSON file defining request matcher + response. The stub definition.	03-mappings/mappings/*.json
Response templating	Handlebars — makes mock responses dynamic ({{now}}, {{request.path}}).	transformers: ["response-template"]
Record/playback	Capture real API traffic and replay offline. Proxy + record.	--proxy-all + --record-mappings
Postman collection	JSON file (v2.1) with requests + pm.test assertions. Portable to Bruno/Insomnia.	postman/*.postman_collection.json
pm.test	Postman assertion: pm.test("Status 200", () => pm.response.to.have.status(200)).	Post-response tab
Breaking change	API change that breaks consumer — field removed, type changed, status changed.	Checklist p30–33, ■
Schema drift	Consumer expects field that provider removed/renamed. Like menu item gone.	Caught by like/regex
Consumer-driven	Consumer writes contract, provider must satisfy. Pact default.	This kit — Pact
Provider-driven	Provider publishes OpenAPI, consumer validates. Alternative to Pact.	OpenAPI + validator
Pact Broker	Server that stores/shares pact files + versions + can-i-deploy. Optional.	At scale, not required here
can-i-deploy	CI gate — checks if consumer/provider versions are compatible per pacts. Blocks breaking deploy.	Provider CI pipeline

What's Next — Create Your Own (3 Steps)

- **1. Fork demo-app** — copy `companion_files/contract-mock-demo-app/` to your project, add your endpoint to `server.js` (Express).
- **2. Add your endpoint to Pact templates** — copy `template-01`, change `path/body/matchers` for your API. Run `npm test` → see pact file.
- **3. Run can-i-deploy in CI** — add `Verifier` to your pipeline, block deploy if verification fails. Use helpers in `contract-mock-practice-lab/helpers/` (`pact-helpers.js`, `wiremock-helpers.js`, `data-factory.js`) to copy.

Free & Open-Source Stack — No Vendor Lock-In. Every tool has a free alternative documented on p3. Pact (MIT), WireMock (Apache 2.0), Express (MIT), Jest (MIT), Postman (free tier). PactFlow / WireMock Cloud are optional commercial extensions, never required.

Realistic Expectations

This kit gives you **runnable templates and a proven workflow** — not magic. You still need to adapt templates to your API, seed provider states for your data, and run Verifier in CI. The checklist catches 14 categories, but your API may have domain-specific breaking changes beyond those. Use the kit as a **starting point**, then extend with your own contracts and mappings. The 5 practice-lab exercises (field removal, type change, required field, templating, UI-against-mock) are designed to train you for exactly that.

Your Action Plan — Next 7 Days

Turn this kit into your own contract suite in one week. Check off each day:

Day	Action	File / Page	Done
Day 1	Read Ch 1–2 + run demo-app (2-terminal workflow)	p6–p11 + companion_files/contract-mock-demo-app/	■
Day 2	Run 5 Pact templates + inspect pacts/WebApp-UserAPI.json	p12–p19 + pact-templates/	■
Day 3	Spin WireMock (Docker or JAR) + test 3 mappings via curl	p20–p24 + wiremock-setup/	■
Day 4	Import Postman collection + run pm.test against WireMock	p25–p26 + postman/	■
Day 5	Wire UI to WireMock via VITE_API_URL + run Playwright E2E	p27–p29 (HERO)	■
Day 6	Run breaking-change checklist on your own API (14 categories)	p30–p33 + checklists/	■
Day 7	Fork demo-app, add your endpoint, write new Pact test, verify	p35 What's Next + helpers/	■

Tip — Start Small

Pick **one endpoint** from your real API (e.g., GET /users/:id). Copy template-01, change path/body/matchers, run npm test. When it passes, add the next endpoint. In 7 days you will have a contract suite for your API.

Notes

Use this space for your API endpoints, matcher choices, and WireMock mapping notes.

@shamshutalks

AI-Native Testing • Genuine Content • Real Results

Instagram: @shamshutalks | YouTube: ShamshuTalks | LinkedIn: ShamshuTalks

Free & Open-Source Stack • No Vendor Lock-In • Pact (MIT) • WireMock (Apache 2.0) • Express (MIT) • Jest (MIT)

API Contract Testing and Mocking Starter Kit for SDETs • ShamshuTalks SDET Series • 35 pages • 2026